

2. Zestawienie połączenia TCP

Znajdź trzy pakiety tworzące TCP three-way handshake.

Jakie flagi występują kolejno?

Który host rozpoczyna połączenie?

Z jakiego portu klienta i do jakiego portu serwera wykonywane jest połączenie?

Filtr: tcp.flags.syn == 1

Wireshark capture showing a TCP three-way handshake. The filter is `tcp.flags.syn == 1`. The packet list shows packets 222, 223, and 337. Packet 222 is a SYN from 192.168.1.226 to 5.181.53.150. Packet 223 is a SYN, ACK from 5.181.53.150 to 192.168.1.226. Packet 337 is a SYN from 192.168.1.226 to 142.250.130.95. The packet details for packet 222 are expanded, showing Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol (Seq=0, Win=65535, Len=0, MSS=1460, WS=256).

Jakie flagi występują kolejno?

- 1. pakiet (nr 222): **SYN**
- 2. pakiet (nr 223): **SYN, ACK**
- 3. pakiet: **ACK**

Który host rozpoczyna połączenie?

- Połączenie rozpoczyna komputer o IP **192.168.1.226**

Z jakiego portu klienta i do jakiego portu serwera wykonywane jest połączenie?

- Port klienta (źródłowy): **52558**
- Port serwera (docelowy): **443**

3. Adresowanie w różnych warstwach OSI

Dla żądania HTTP odczytaj:

źródłowy i docelowy adres MAC,

źródłowy i docelowy adres IP,
źródłowy i docelowy port TCP.

Wyjaśnij, jaką rolę pełni każdy z tych identyfikatorów oraz dlaczego adres MAC wskazuje urządzenie na lokalnym łączu, a adres IP — końcowego odbiorcę pakietu.

Filtr: http.request

*Ethernet						
Plik Edycja Widok Przejdź Przechwytywanie Analiza Statystyki Telefonía Bezprzewodowe Narzędzia Pomoc						
http.request						
No.	Time	Source	Destination	Protocol	Length	Info
11	1.246360800	192.168.1.102	239.255.255.250	SSDP	179	M-SEARCH * HTTP/1.1
45894	467.415363800	192.168.1.226	34.223.124.45	HTTP	405	GET / HTTP/1.1
46022	471.463143500	192.168.1.226	34.223.124.45	HTTP	380	GET / HTTP/1.1
47942	507.945059500	192.168.1.226	34.223.124.45	HTTP	467	GET /online HTTP/1.1
47946	508.144354700	192.168.1.226	34.223.124.45	HTTP	468	GET /online/ HTTP/1.1
47950	508.377226700	192.168.1.226	34.223.124.45	HTTP	477	GET /favicon.ico HTTP/1.1

> Frame 11: Packet, 179 bytes on wire (1432 bits), 179 bytes captured	0000 01 00 5e 7f ff fa a8 a1 59 28 11 22 08 00 45 00	..^.....Y(."
> Ethernet II, Src: ASRockIncorp_28:11:22 (a8:a1:59:28:11:22), Dst: IP	0010 00 a5 4b 41 00 00 04 11 b8 fe c0 a8 01 66 ef ff	..KA.....
> Internet Protocol Version 4, Src: 192.168.1.102, Dst: 239.255.255.250	0020 ff fa e3 05 07 6c 00 91 70 33 4d 2d 53 45 41 52	1..p3M-S
> User Datagram Protocol, Src Port: 58117, Dst Port: 1900	0030 43 48 20 2a 20 48 54 54 50 2f 31 2e 31 0d 0a 48	CH * HTTP/1.1
> Simple Service Discovery Protocol	0040 6f 73 74 3a 20 32 33 39 2e 32 35 35 2e 32 35 35	ost: 239 .255.
	0050 2e 32 35 30 3a 31 39 30 30 0d 0a 53 54 3a 20 75	.250:190 0..S
	0060 72 6e 3a 73 63 68 65 6d 61 73 2d 75 70 6e 70 2d	rn:schem as-up
	0070 6f 72 67 3a 64 65 76 69 63 65 3a 49 6e 74 65 72	org:devi ce:Ir
	0080 6e 65 74 47 61 74 65 77 61 79 44 65 76 69 63 65	netGatew ayDev
	0090 3a 31 0d 0a 4d 61 6e 3a 20 22 73 73 64 70 3a 64	:1..Man: "ssc
	00a0 69 73 63 6f 76 65 72 22 0d 0a 4d 58 3a 20 33 0d	iscover" ..MX:
	00b0 0a 0d 0a	...

Pakiet 47942

Wireshark · Pakiet 47942 · Ethernet	
> Frame 47942: Packet, 467 bytes on wire (3736 bits), 467 bytes captured (3736 bits) on interface \Device\NPF_{...}	
> Ethernet II, Src: LCFCElectron_0d:98:5d (84:a9:38:0d:98:5d), Dst: zte_8d:8f:a0 (d4:72:26:8d:8f:a0)	
> Internet Protocol Version 4, Src: 192.168.1.226, Dst: 34.223.124.45	
> Transmission Control Protocol, Src Port: 50982, Dst Port: 80, Seq: 1, Ack: 1, Len: 413	
> Hypertext Transfer Protocol	
0000 d4 72 26 8d 8f a0 84 a9 38 0d 98 5d 08 00 45 00	·r&·.....8..]·E·
0010 01 c5 6c 6e 40 00 80 06 00 00 c0 a8 01 e2 22 df	·ln@·....."
0020 7c 2d c7 26 00 50 f0 de 1a ad c2 04 90 6f 50 18	·-&·P·.....oP·
0030 ff ff 63 4e 00 00 47 45 54 20 2f 6f 6e 6c 69 6e	·cN·GE T /onlin
0040 65 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74	e HTTP/1 .1·Host
0050 3a 20 77 6f 6e 64 65 72 66 75 6c 66 75 6e 67 6c	: wonder fulfungl
0060 6f 77 69 6e 67 73 74 61 72 73 2e 6e 65 76 65 72	owingsta rs.never
0070 73 73 6c 2e 63 6f 6d 0d 0a 55 73 65 72 2d 41 67	ssl.com· User-Ag
0080 65 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30	ent: Moz illa/5.0
0090 20 28 57 69 6e 64 6f 77 73 20 4e 54 20 31 30 2e	(Window s NT 10.
00a0 30 3b 20 57 69 6e 36 34 3b 20 78 36 34 3b 20 72	0; Win64 ; x64; r
00b0 76 3a 31 35 34 2e 30 29 20 47 65 63 6b 6f 2f 32	v:154.0) Gecko/2
00c0 30 31 30 30 31 30 31 20 46 69 72 65 66 6f 78 2f	0100101 Firefox/
No.: 47942 · Time: 507.945059500 · Source: 192.168.1.226 · Destination: 34.223.124.45 · Protocol: HTTP · Length: 467 · Info: GET /online HTTP/1.1	

Adres MAC:

- Źródłowy: `84:a9:38:0d:98:5d` (*Twój komputer*)
- Docelowy: `d4:72:26:8d:8f:a0` (*Twój router*)

Adres IP:

- Źródłowy: `192.168.1.226` (*Twój komputer*)
- Docelowy: `34.223.124.45` (*Serwer HTTP*)

Port TCP:

- Źródłowy: `50982`
- Docelowy: `80`

Wyjaśnienie ról:

- **Adres MAC (Warstwa 2):** Identyfikuje urządzenie tylko w lokalnej sieci (LAN). Na każdym routerze jest podmieniany na nowy, bo służy tylko do skoku do kolejnego urządzenia na drodze.
- **Adres IP (Warstwa 3):** Identyfikuje końcowy cel w internecie (jak adres na kopercie listu). Nie zmienia się przez całą drogę od nadawcy do odbiorcy.
- **Port TCP (Warstwa 4):** Wskazuje konkretną aplikację na komputerze (np. przeglądarkę lub serwer WWW).

4. Analiza żądania HTTP

Otwórz żądanie HTTP i ustal:

jaka metoda została użyta,
jaki zasób pobiera klient,
jaka jest wersja HTTP,
jaka domena znajduje się w nagłówku Host,
czy treść żądania jest zaszyfrowana.
Filtr: `http.request`

Metoda: `GET`

Zasób: `/online`

Wersja HTTP: `HTTP/1.1`

Domena w Host: `wonder-fulfunglowingstars.neverssl.com`

Szyfrowanie: Nie, czytelny tekst (plain text).

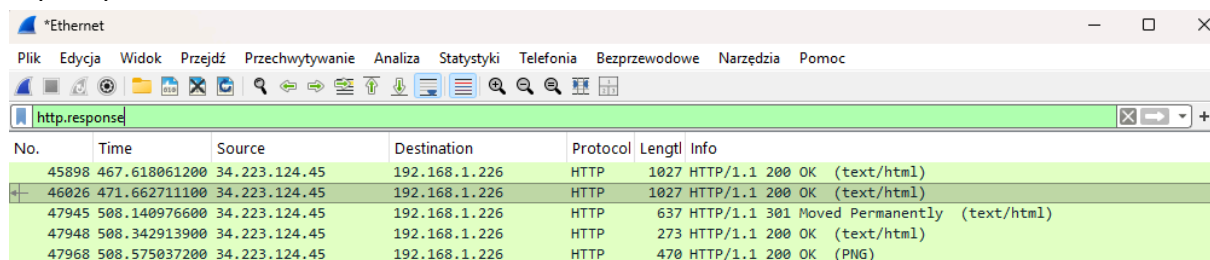
5. Odpowiedź HTTP i zakończenie połączenia

Znajdź odpowiedź serwera i ustal:

jaki kod odpowiedzi zwrócił serwer i co on oznacza,
co znajduje się w treści odpowiedzi,
które pakiety kończą połączenie TCP i jakie mają flagi.

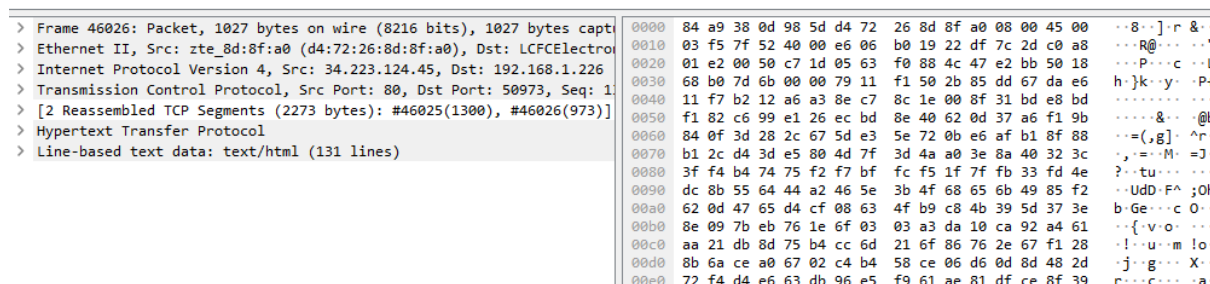
Filtry: `http.response` oraz `tcp.flags.fin == 1`

`http.response`



Wireshark capture showing HTTP responses filtered by 'http.response'. The table lists packets with their time, source, destination, protocol, length, and information.

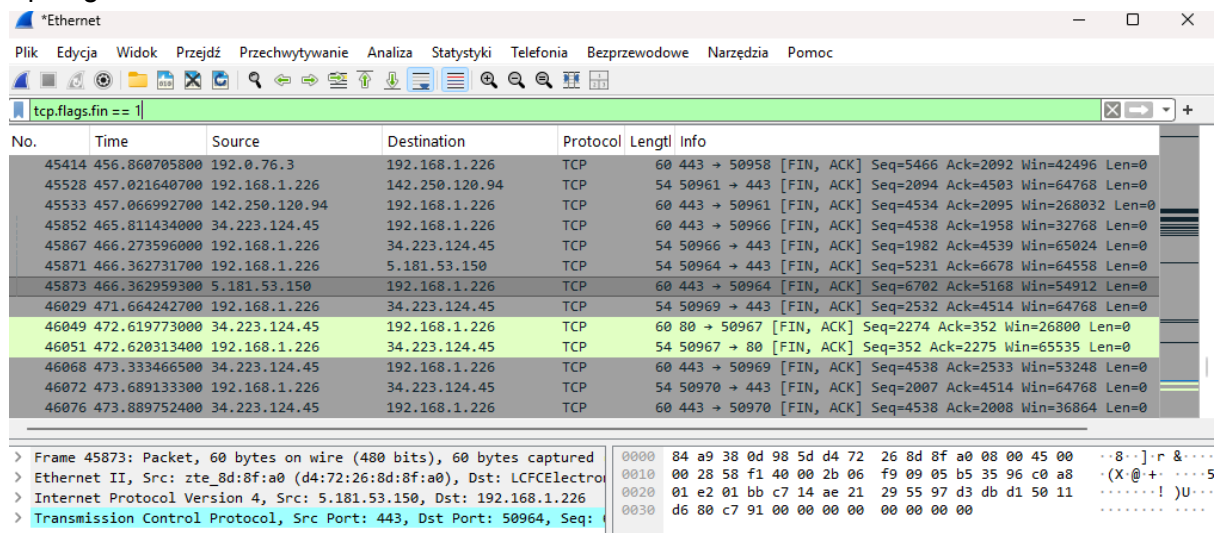
No.	Time	Source	Destination	Protocol	Length	Info
45898	467.618061200	34.223.124.45	192.168.1.226	HTTP	1027	HTTP/1.1 200 OK (text/html)
46026	471.662711100	34.223.124.45	192.168.1.226	HTTP	1027	HTTP/1.1 200 OK (text/html)
47945	508.140976600	34.223.124.45	192.168.1.226	HTTP	637	HTTP/1.1 301 Moved Permanently (text/html)
47948	508.342913900	34.223.124.45	192.168.1.226	HTTP	273	HTTP/1.1 200 OK (text/html)
47968	508.575037200	34.223.124.45	192.168.1.226	HTTP	470	HTTP/1.1 200 OK (PNG)



Wireshark packet details and hex view for packet 46026. The details pane shows the packet structure, and the hex view shows the raw data.

Details	Hex View
> Frame 46026: Packet, 1027 bytes on wire (8216 bits), 1027 bytes captured	0000 84 a9 38 0d 98 5d d4 72 26 8d 8f a0 08 00 45 00 ...8...].r &...
> Ethernet II, Src: zte_8d:8f:a0 (d4:72:26:8d:8f:a0), Dst: LCFCElectro	0010 03 f5 7f 52 40 00 e6 06 b0 19 22 df 7c 2d c0 a8 ...R@... ..
> Internet Protocol Version 4, Src: 34.223.124.45, Dst: 192.168.1.226	0020 01 e2 00 50 c7 1d 05 63 f0 88 4c 47 e2 bb 50 18 ...P...c...l
> Transmission Control Protocol, Src Port: 80, Dst Port: 50973, Seq: 1	0030 68 b0 7d 6b 00 00 79 11 f1 50 2b 85 dd 67 da e6 h...k...y...P
> [2 Reassembled TCP Segments (2273 bytes): #46025(1300), #46026(973)]	0040 11 f7 b2 12 a6 a3 8e c7 8c 1e 00 8f 31 bd e8 bd
> Hypertext Transfer Protocol	0050 f1 82 c6 99 e1 26 ec bd 8e 40 62 0d 37 a6 f1 9b&...@l
> Line-based text data: text/html (131 lines)	0060 84 0f 3d 28 2c 67 5d e3 5e 72 0b e6 af b1 8f 88 ...=(,g].^r
	0070 b1 2c d4 3d e5 80 4d 7f 3d 4a a0 3e 8a 40 32 3c ...=.M.=J
	0080 3f f4 b4 74 75 f2 f7 bf fc f5 1f 7f fb 33 fd 4e ?...tu... ..
	0090 dc 8b 55 64 44 a2 46 5e 3b 4f 68 65 6b 49 85 f2 ...UdD.F^;Ol
	00a0 62 0d 47 65 d4 cf 08 63 4f b9 c8 4b 39 5d 37 3e b.Ge...c O
	00b0 8e 09 7b eb 76 1e 6f 03 03 a3 da 10 ca 92 a4 61 ...{.v.o... ..
	00c0 aa 21 db 8d 75 b4 cc 6d 21 6f 86 76 2e 67 f1 28 !...u...m !o
	00d0 8b 6a ce a0 67 02 c4 b4 58 ce 06 d6 0d 8d 48 2d j...g...X
	00e0 72 f4 d4 e6 63 db 96 e5 f9 61 ae 81 df ce 8f 39 r...c...a

`tcp.flags.fin == 1`



Wireshark capture showing TCP segments filtered by 'tcp.flags.fin == 1'. The table lists packets with their time, source, destination, protocol, length, and information.

No.	Time	Source	Destination	Protocol	Length	Info
45414	456.860705800	192.0.76.3	192.168.1.226	TCP	60	443 → 50958 [FIN, ACK] Seq=5466 Ack=2092 Win=42496 Len=0
45528	457.021640700	192.168.1.226	142.250.120.94	TCP	54	50961 → 443 [FIN, ACK] Seq=2094 Ack=4503 Win=64768 Len=0
45533	457.066992700	142.250.120.94	192.168.1.226	TCP	60	443 → 50961 [FIN, ACK] Seq=4534 Ack=2095 Win=268032 Len=0
45852	465.811434000	34.223.124.45	192.168.1.226	TCP	60	443 → 50966 [FIN, ACK] Seq=4538 Ack=1958 Win=32768 Len=0
45867	466.273596000	192.168.1.226	34.223.124.45	TCP	54	50966 → 443 [FIN, ACK] Seq=1982 Ack=4539 Win=65024 Len=0
45871	466.362731700	192.168.1.226	5.181.53.150	TCP	54	50964 → 443 [FIN, ACK] Seq=5231 Ack=6678 Win=64558 Len=0
45873	466.362959300	192.168.1.226	192.168.1.226	TCP	60	443 → 50964 [FIN, ACK] Seq=6702 Ack=5168 Win=54912 Len=0
46029	471.664242700	192.168.1.226	34.223.124.45	TCP	54	50969 → 443 [FIN, ACK] Seq=2532 Ack=4514 Win=64768 Len=0
46049	472.619773000	34.223.124.45	192.168.1.226	TCP	60	80 → 50967 [FIN, ACK] Seq=2274 Ack=352 Win=26800 Len=0
46051	472.620313400	192.168.1.226	34.223.124.45	TCP	54	50967 → 80 [FIN, ACK] Seq=352 Ack=2275 Win=65535 Len=0
46068	473.333466500	34.223.124.45	192.168.1.226	TCP	60	443 → 50969 [FIN, ACK] Seq=4538 Ack=2533 Win=53248 Len=0
46072	473.689133300	192.168.1.226	34.223.124.45	TCP	54	50970 → 443 [FIN, ACK] Seq=2007 Ack=4514 Win=64768 Len=0
46076	473.889752400	34.223.124.45	192.168.1.226	TCP	60	443 → 50970 [FIN, ACK] Seq=4538 Ack=2008 Win=36864 Len=0

Packet details for packet 45873:

- > Frame 45873: Packet, 60 bytes on wire (480 bits), 60 bytes captured
- > Ethernet II, Src: zte_8d:8f:a0 (d4:72:26:8d:8f:a0), Dst: LCFCElectro
- > Internet Protocol Version 4, Src: 5.181.53.150, Dst: 192.168.1.226
- > Transmission Control Protocol, Src Port: 443, Dst Port: 50964, Seq: 6702, Ack: 5168, Win: 54912, Len: 0

Hex view for packet 45873:

```
0000 84 a9 38 0d 98 5d d4 72 26 8d 8f a0 08 00 45 00 ...8...].r &...
0010 00 28 58 f1 40 00 2b 06 f9 09 05 b5 35 96 c0 a8 ... (X@+ .....5
0020 01 e2 01 bb c7 14 ae 21 29 55 97 d3 db d1 50 11 .....!)U...
0030 d6 80 c7 91 00 00 00 00 00 00 00 00 00 00 00 00 ..... ..
```

Kod odpowiedzi serwera i jego znaczenie:

- **Kod:** 200 OK
- **Znaczenie:** Żądanie powiodło się – serwer odnalazł i poprawnie zwrócił żądany zasób.

Treść odpowiedzi:

- W treści odpowiedzi znajduje się kod źródłowy strony internetowej w formacie **HTML** (`text/html`) oraz pliki graficzne (np. w formacie **PNG**).

Pakiety kończące połączenie TCP i ich flagi:

- **Numeracja pakietów:** Pakiety nr 46049 oraz 46051.
- **Występujące flagi:** Oba pakiety posiadają włączone flagi **FIN**, **ACK** (służące do zasygnalizowania zamknięcia transmisji oraz potwierdzenia odbioru danych).

Laboratorium: identyfikacja warstw w Wiresharku

Cel ćwiczenia

Student potrafi wskazać w realnej komunikacji:

- 🟢 ramkę Ethernet lub Wi-Fi
- 🟢 pakiet IPv4/IPv6
- 🟢 TCP/UDP/ICMP
- 🟢 TLS lub HTTP

Ruch do wygenerowania

ping 8.8.8.8 oraz curl https://example.com

Filtry startowe

icmp & tcp.port == 443 & !is & http

No.	Time	Source	Destination	Protocol	Length	Info
→ 228	9.674239500	192.168.1.226	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 2...
← 229	9.708227000	8.8.8.8	192.168.1.226	ICMP	74	Echo (ping) reply id=0x0001, seq=1/256, ttl=113 (request in...
248	10.685512500	192.168.1.226	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 2...
250	10.719464600	8.8.8.8	192.168.1.226	ICMP	74	Echo (ping) reply id=0x0001, seq=2/512, ttl=113 (request in...
265	11.705253100	192.168.1.226	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 2...
266	11.734493100	8.8.8.8	192.168.1.226	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=113 (request in...
300	12.720848900	192.168.1.226	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in ...
301	12.751420600	8.8.8.8	192.168.1.226	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=113 (request in...

> Frame 228: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0

> Ethernet II, Src: LCFCElectron_0d:98:5d (84:a9:38:0d:98:5d), Dst: zt...

> Internet Protocol Version 4, Src: 192.168.1.226, Dst: 8.8.8.8

> Internet Control Message Protocol

0000 d4 72 26 8d 8f a0 84 a9 38 0d 98 5d 08 00 45 00 ..r&.....8..]

0010 00 3c 9e 4f 00 00 80 01 00 00 c0 a8 01 e2 08 08 <<O.....

0020 08 08 08 00 4d 5a 00 01 00 01 61 62 63 64 65 66MZ....abc

0030 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 ghijklmn opqrs

0040 77 61 62 63 64 65 66 67 68 69 wabcdefg hi

No.	Time	Source	Destination	Protocol	Length	Info
187	7.684677400	192.168.1.226	104.18.39.21	TLSv1.2	82	Application Data
205	8.123589700	192.168.1.226	5.181.53.150	TLSv1.3	189	Application Data
206	8.123638400	192.168.1.226	5.181.53.150	TLSv1.3	1551	Application Data
207	8.123754700	192.168.1.226	5.181.53.150	TLSv1.3	765	Application Data
209	8.180847600	5.181.53.150	192.168.1.226	TLSv1.3	89	Application Data
211	8.183838700	5.181.53.150	192.168.1.226	TLSv1.3	312	Application Data
222	8.877943000	20.86.94.139	192.168.1.226	TLSv1.2	81	Application Data
230	9.852077600	104.18.8.126	192.168.1.226	TLSv1.2	78	Application Data
231	9.852325600	192.168.1.226	104.18.8.126	TLSv1.2	82	Application Data
237	9.984334200	192.168.1.226	3.67.245.95	TLSv1.2	108	Application Data
239	10.029289500	3.67.245.95	192.168.1.226	TLSv1.2	110	Application Data
241	10.110721200	104.18.8.126	192.168.1.226	TLSv1.2	78	Application Data
242	10.110857500	192.168.1.226	104.18.8.126	TLSv1.2	82	Application Data

> Frame 222: Packet, 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface 0	0000	84 a9 38 0d 98 5d d4 72 26 8d 8f a0 08 00 45 00	..8..].r &..
> Ethernet II, Src: zte_8d:8f:a0 (d4:72:26:8d:8f:a0), Dst: LCFCElectro	0010	00 43 f5 40 40 00 2a 06 26 09 14 56 5e 8b c0 a8	C@@*.* &..
> Internet Protocol Version 4, Src: 20.86.94.139, Dst: 192.168.1.226	0020	01 e2 01 bb d1 96 df 2b 79 6a df 64 8d 95 50 18	+ yj
> Transmission Control Protocol, Src Port: 443, Dst Port: 53654, Seq:	0030	16 3c a2 49 00 00 17 03 03 00 16 9c 68 4e 0d 7d	<I.....
> Transport Layer Security	0040	eb 79 bc 90 f2 d8 b8 87 2e db 14 52 07 ec dc ee	y.....
	0050	07	.

- **Generowanie ruchu:**
 - **ping 8.8.8.8** – wygenerował ruch diagnostyczny w warstwie sieciowej (ICMP).
 - **curl https://google.com** – wygenerował szyfrowany ruch w warstwie aplikacji/prezentacji (TLS/TCP na porcie 443).
- **1. Warstwa 2 – Łącza danych (Ethernet II / Wi-Fi):**
 - **Filtr w Wiresharku:** **icmp** lub **tls**
 - **Nazwa sekcji:** **Ethernet II**
 - **Odczytane dane:** Adres MAC źródłowy (**84:a9:38:0d:98:5d**) oraz docelowy MAC bramy domyślnej (**d4:72:26:8d:8f:a0**).
 - **Rola:** Identyfikuje fizyczne urządzenia w ramach lokalnej sieci (LAN).
- **2. Warstwa 3 – Sieciowa (IPv4 / ICMP):**
 - **Filtr w Wiresharku:** **icmp**
 - **Nazwa sekcji:** **Internet Protocol Version 4** oraz **Internet Control Message Protocol**
 - **Odczytane dane:** Adres IP źródłowy (**192.168.1.226**) i docelowy IP (**8.8.8.8**).
 - **Rola:** Odpowiada za adresowanie globalne i trasowanie pakietów między różnymi sieciami w Internecie.
- **3. Warstwa 4 – Transportowa (TCP / UDP):**
 - **Filtr w Wiresharku:** **tcp.port == 443**
 - **Nazwa sekcji:** **Transmission Control Protocol**
 - **Odczytane dane:** Port źródłowy klienta (**53654**) oraz port docelowy serwera (**443**).
 - **Rola:** Zapewnia niezawodną transmisję danych między konkretnymi procesami/aplikacjami na hostach.
- **4. Warstwa 6/7 – Prezentacji i Aplikacji (TLS / HTTP):**
 - **Filtr w Wiresharku:** **tls**

- **Nazwa sekcji:** Transport Layer Security
- **Odczytane dane:** Rekordy Application Data (szyfrowana treść protokołu HTTPS).
- **Rola:** Odpowiada za bezpośrednią obsługę aplikacji użytkownika oraz bezpieczne (szyfrowane) szyfrowanie przesyłanych danych.